```
it "doesn't allow  creation of a task without a size" do
  creator = CreatesProject.new(name: "Test", task_string: "size:no_size")
  creator.create
  expect(creator.project.tasks.map(&:title)).to eq(["size"])
end
```

In this test, you're validating that the second task, with the clever name no size, does not get added to the project. This test works whether the size limitation is implemented as a Rails validation or as some kind of filter that the CreatesProject class manages. Again, you're testing the behavior, not the implementation. This strategy works for all kinds of Rails validations, including uniqueness (create two objects and validate that the second one doesn't save). When I create a custom validator, though, as either a method or a separate class, it goes through the same TDD process as any other method.

Testing ActiveRecord Finders

ActiveRecord provides a rich set of methods that are wrappers around SQL statements sent to your database. These methods are collectively referred to as *finders*. One great feature of ActiveRecord finders is that they can be composed, allowing you to express a compound statement like "bring me the most recently completed five large tasks" as Task.where(status: completed).order("completed_at DESC").where("size > 3").limit(5).

You can even compose the finders if you extract them to their own methods in pieces:

```
class Task < ActiveRecord::Base

  def self.completed
    where(status: :completed)
  end

  def self.large
    where("size > 3")
  end

  def self.most_recent
    order("completed_at DESC")
  end

  def self.recent_done_and_large
    completed.large.most_recent.limit(5)
  end
end
```

Being able to compose this logic is awesome. But finder methods occupy an awkward place between methods you might write and Rails core features, leading to the question of how best to test them.